# AI-POWERED RESILIENT SUPPLY CHAIN INTELLIGENCE SYSTEM

Tech Stack, Architecture & Lovable Prompt

*Perishable-Aware Logistics Intelligence — Avocado Use Case*

| | |
|---|---|
| **Version** | 1.0 — Hackathon Edition |
| **Target Timeline** | 48–72 Hours (MVP) |
| **Core Stack** | React + FastAPI + Google Cloud + Gemini AI |
| **Demo Focus** | Avocado Shipment: Kenya → India |
| **AI Integration** | Gemini Pro, Vertex AI, scikit-learn |

> *This document defines the complete technology choices, alternatives, AI/LLM integration strategy, and a production-ready Lovable prompt for building a working demo of the Supply Chain Intelligence System in a hackathon setting.*

# 1. Executive Summary & System Overview

The AI-Powered Resilient Supply Chain Intelligence System is a real-time, AI-driven logistics platform that goes beyond static dashboards. It continuously monitors shipments, predicts disruptions before they cascade, models quality degradation for perishable goods, and uses Google Gemini to explain every decision in plain English.

**Core workflow:**

- TRACK — Shipment state tracked across a graph of nodes/edges
- ANALYZE — Traffic, weather, distance inputs processed in real-time
- PREDICT — Risk score + delay probability computed per route
- SIMULATE — Downstream domino effects modeled
- DECIDE — Multi-objective optimizer selects best route
- EXPLAIN — Gemini generates natural language reasoning

> *Design philosophy: Maximum AI automation, minimum manual intervention. Every layer uses AI — from risk prediction to route explanation to UI generation.*

# 2. Full System Architecture

## 2.1 Architecture Overview — Layer by Layer

The system is structured into 7 distinct layers, each responsible for a specific function. All layers communicate via REST APIs and share state through Firestore.

| Layer | Name | Function | Primary Tech |
|---|---|---|---|
| L1 | Data Ingestion | Collect shipment, traffic, weather inputs | FastAPI + Google Maps API |
| L2 | Graph Tracking | Model supply chain as weighted graph | NetworkX + Firestore |
| L3 | Risk Prediction | Compute delay probability + risk score | scikit-learn (RF) + rule engine |
| L4 | Quality Engine | Predict avocado degradation over time+delay | Custom decay formula + Gemini |
| L5 | Route Optimizer | Dijkstra/A* with multi-objective weights | NetworkX + custom scorer |
| L6 | Decision Engine | Combine risk+quality → reroute decision | Rule-based logic + LLM assist |
| L7 | Explainability | Natural language explanation of decisions | Google Gemini Pro API |

## 2.2 Data Flow Diagram (Textual)

> *User Input → [FastAPI Backend] → Google Maps API (routes) + Weather API (conditions) → Risk Engine + Quality Engine → Graph Optimizer → Decision Engine → Gemini Explanation → Frontend (React) → Firestore (persist state)*

## 2.3 Backend Folder Structure

```
supply-chain-ai/
├── backend/
│   ├── main.py                 # FastAPI entry point
│   ├── routers/
│   │   ├── risk.py             # /predict-risk
│   │   ├── quality.py          # /predict-quality
│   │   ├── routing.py          # /optimize-route
│   │   └── explain.py          # /explain (Gemini)
│   ├── models/
│   │   ├── risk_model.pkl      # Trained RF model
│   │   └── schemas.py          # Pydantic models
│   ├── services/
│   │   ├── graph.py            # NetworkX graph
│   │   ├── gmaps.py            # Google Maps client
```

```
│   │   ├── gemini.py            # Gemini API client
│   │   └── firestore.py         # DB operations
│   └── Dockerfile
├── frontend/                    # React (Vite)
│   └── src/components/
│       ├── MapView.jsx
│       ├── QualityChart.jsx
│       ├── RiskPanel.jsx
│       └── GeminiExplain.jsx
└── infrastructure/
    ├── cloudbuild.yaml
    └── firebase.json
```

# 3. Technology Stack — Chosen Tools & Why

## 3.1 Frontend

| Component | Chosen Tool | Why Chosen | Alternatives |
|---|---|---|---|
| UI Framework | React + Vite | Fastest DX; huge ecosystem; ideal for real-time state updates | Next.js, Vue.js, Svelte |
| Styling | Tailwind CSS | Utility-first, no context switching, fast UI iteration in hackathon | Material UI, Chakra UI, Bootstrap |
| Maps | Google Maps JS API | Hard requirement; supports custom overlays, routes, polylines | Mapbox, Leaflet, HERE Maps |
| Charts | Recharts | React-native, declarative, easy quality/risk time-series charts | Chart.js, Plotly, Victory |
| State Management | Zustand | Minimal boilerplate vs Redux; perfect for real-time updates | Redux, Jotai, React Query |
| Real-time Updates | Firebase SDK (Firestore listeners) | Native Firestore onSnapshot; zero polling needed | WebSockets, SSE, SWR |

## 3.2 Backend

| Component | Chosen Tool | Why Chosen | Alternatives |
|---|---|---|---|
| API Framework | FastAPI (Python) | Auto OpenAPI docs, async support, perfect ML/Python integration | Node.js + Express, Django REST |
| Graph Engine | NetworkX | Python-native; Dijkstra/A* built-in; easy edge weight customization | iGraph, custom BFS/DFS |
| ML / Risk Model | scikit-learn (Random Forest) | Fast training; interpretable; no GPU needed; pickle-serializable | XGBoost, LightGBM, PyTorch |
| Data Validation | Pydantic v2 | Built into FastAPI; enforces strict API contracts | Marshmallow, Cerberus |
| Maps/Routing | googlemaps Python SDK | Official SDK; distance matrix + directions in 2 lines | requests + raw API |
| Task Queue | Background Tasks (FastAPI) | No extra infra; sufficient for hackathon async calls | Celery + Redis, Cloud Tasks |

## 3.3 AI / LLM Layer

| Function | Chosen Tool | Why Chosen | Alternatives |
|---|---|---|---|
| Decision Explanation | Gemini Pro (AI Studio) | Hard requirement; fastest integration via API key; multilingual | GPT-4o, Claude, Llama 3 |
| Risk Classification | Random Forest + Gemini | RF for structured data; Gemini enriches with context | Logistic Regression, XGBoost |
| Quality Modeling | Decay Formula + Gemini | Deterministic formula; Gemini adds perishability context | LSTM, ARIMA (overkill for MVP) |
| Route Reasoning | Gemini Pro | Summarizes 2-route comparison in natural language | Custom NLG templates |
| Scenario Insights | Gemini Pro (chain) | Multi-turn chat for interactive scenario exploration | LangChain + any LLM |
| Structured Output | Gemini JSON mode | Forces valid JSON for route/risk data extraction | Function calling, instructor lib |

## 3.4 Google Cloud / Infrastructure

| Service | Chosen Tool | Why Chosen | Alternatives |
|---|---|---|---|
| Backend Hosting | Cloud Run | Serverless; Docker-native; auto-scales to zero cost | GKE, App Engine, Cloud Functions |
| Frontend Hosting | Firebase Hosting | CDN-backed; instant deploys; HTTPS out of box | Vercel, Netlify, GCS Bucket |
| Database | Firestore | Real-time listeners; NoSQL flexibility; Firebase SDK | Cloud SQL, BigQuery, Redis |
| CI/CD | Cloud Build | Native GCP; triggers on push; builds + deploys Cloud Run | GitHub Actions, Jenkins |
| Maps + Traffic | Google Maps Platform | Directions API + Distance Matrix + real-time traffic | HERE Maps, TomTom |
| Model Registry | Vertex AI (optional) | For productionizing RF model beyond hackathon | MLflow, BentoML |

# 4. Module Deep Dives — Tech Choices per Feature

## 4.1 Data Ingestion Layer

The ingestion layer is the entry point for all real-time signals. We use a hybrid of mock data and real APIs to ensure the MVP works even without API credits.

- Shipment Data: JSON payload via POST /shipments — includes source, destination, cargo_type, deadline, weight_kg
- Traffic: Google Maps Directions API — fetches real-time ETA and distance for each route segment
- Weather: OpenWeatherMap API (free tier) — fetches condition + wind + visibility per waypoint
- Mock Generator: Python Faker + random disruption injection — allows offline demo without API calls

> *AI Automation: Gemini can be prompted to generate realistic mock shipment scenarios from just a city pair and cargo type. Zero manual data entry needed for demos.*

## 4.2 Graph-Based Tracking System

The entire supply chain is modeled as a directed weighted graph using NetworkX. This enables algorithm-level route comparison and simulation.

- Nodes: Ports, warehouses, distribution hubs, customs checkpoints
- Edges: Transport legs (sea, air, road) with attributes: distance_km, base_time_hrs, risk_score, quality_loss_per_hr
- Storage: Graph structure serialized to Firestore; edge weights updated dynamically on each prediction cycle
- Visualization: Google Maps polylines drawn per route; node markers color-coded by risk level (green/yellow/red)

> *Why NetworkX over custom implementation: Dijkstra, A*, Bellman-Ford all available as one-liners. Saves 4-6 hours of hackathon time.*

## 4.3 Risk Prediction Engine

Two-tier risk system: fast rule-based scoring for real-time feedback, and a scikit-learn Random Forest for nuanced predictions. Both run on the same input vector.

| Aspect | Rule-Based Engine | ML Engine (Random Forest) |
|---|---|---|
| Speed | < 1ms (instant) | 50-100ms (model inference) |

| Aspect | Rule-Based Engine | ML Engine (Random Forest) |
|---|---|---|
| Inputs | weather_score, congestion_level | 12 features incl. day_of_week, port_history |
| Output | risk: low/medium/high | delay_prob: 0.0–1.0, risk_score: 0–100 |
| Training Data | Not needed | 500 mock records (pre-generated) |
| Hackathon Use | Primary (always on) | Secondary (enriches explanation) |

**Risk Score Formula (Rule-Based):**

```
risk_score = (weather_weight * weather_score) +
             (traffic_weight * congestion_level) +
             (distance_factor * normalized_distance)
 # Weights: weather=0.35, traffic=0.40, distance=0.25
```

## 4.4 Quality Prediction Engine (Perishable Intelligence)

The most novel module — models avocado quality degradation as a function of time, temperature deviation, and accumulated delay. This is what differentiates the system from generic route planners.

**Core Decay Formula:**

```
# Base model
 quality = 100 - (elapsed_hours + delay_hours) * degradation_rate
  # Temperature-adjusted model (advanced)
 temp_factor = 1 + max(0, (temp_celsius - optimal_temp) * 0.05)
 quality = 100 - (elapsed_hours + delay_hours) * degradation_rate *
temp_factor
  # Status thresholds
 status = 'fresh' if quality > 75 else 'acceptable' if quality > 40
else 'spoiled'
```

**Avocado-specific parameters:**

- optimal_temp: 7°C (cold chain standard)
- degradation_rate: 1.2 quality points/hour at optimal temp
- Shelf life at optimal: ~83 hours (3.5 days)
- Each 5°C above optimal: +5% degradation multiplier

> *AI Enhancement: Gemini receives the quality score, elapsed time, and route comparison, then generates a perishability risk narrative including economic loss estimates. This is the 'wow factor' of the demo.*

## 4.5 Route Optimization Engine

Dijkstra's algorithm is used as the baseline with A* as an enhancement. Edge weights are a composite score combining travel time, risk, and quality impact. This ensures the system recommends the route that preserves the most value, not just the fastest one.

**Composite Edge Weight:**

```
edge_weight = (w1 * normalized_time) +
              (w2 * risk_score) +
              (w3 * quality_loss_factor)
 # Weights: time=0.3, risk=0.4, quality=0.3
 # For perishables, quality weight increases to 0.5
```

- Primary Route: Lowest composite score (Dijkstra shortest path)
- Alternative Route: 2nd shortest path using K-shortest paths (Yen's algorithm)
- Emergency Reroute: Triggered when quality < 40 or risk_score > 75

## 4.6 Impact Simulation Engine

Models the domino effect of a delay at any node. For hackathon MVP, this is a simplified dependency graph where each node knows which downstream nodes depend on it.

- Dependency graph stored in Firestore as node → [dependent_nodes] mapping
- When delay is injected, BFS traversal calculates cascading impact count
- Output: 'Delay at Mombasa Port impacts 3 downstream warehouses and 2 retail operations'

> *Gemini Integration: The impact simulation output is fed directly to Gemini to generate a human-readable supply chain impact narrative, quantifying the economic and operational consequences.*

## 4.7 Decision Engine

The decision engine is a multi-objective policy layer. It combines risk, quality, and time scores into a final action recommendation using predefined thresholds with Gemini as an optional advisor for edge cases.

| Condition | Quality Score | Risk Score | Decision |
|---|---|---|---|
| All clear | > 75 | < 40 | Continue on current route |
| Quality warning | 40–75 | Any | Alert + monitor closely |

| Condition | Quality Score | Risk Score | Decision |
|---|---|---|---|
| Risk spike | Any | 60–80 | Prepare alternate route |
| Critical — reroute | < 40 OR | > 80 | AUTO-REROUTE immediately |
| Spoilage imminent | < 20 | > 60 | Emergency route + alert buyer |

# 5. API Design — Endpoints & Contracts

| Method | Endpoint | Request Body (key fields) | Response (key fields) |
|---|---|---|---|
| POST | /predict-risk | route_id, distance_km, weather_score, congestion_level | risk_score (0-100), delay_prob (0-1), risk_level |
| POST | /predict-quality | cargo_type, elapsed_hours, delay_hours, temp_celsius | quality_score (0-100), status, economic_loss_pct |
| POST | /optimize-route | origin, destination, cargo_type, deadline_hours | primary_route, alt_route, recommendation |
| POST | /explain | route_comparison, risk_data, quality_data, decision | explanation (string), confidence, key_factors[] |
| POST | /inject-disruption | shipment_id, disruption_type, duration_hrs | updated_risk, updated_quality, new_decision |
| GET | /shipments/{id} | (path param) | full shipment state, current route, history |
| POST | /simulate-impact | disrupted_node_id, delay_hours | affected_nodes[], estimated_delay, economic_impact |

# 6. AI & LLM Integration — Maximum Automation

## 6.1 Where AI/LLMs Are Used (Full Map)

| Module | AI Tool | Automation Level | Human Role |
|---|---|---|---|
| Decision Explanation | Gemini Pro | Full — auto-generated | Read output |
| Mock Data Generation | Gemini Pro | Full — from city pair prompt | None |
| Risk Classification | Random Forest + Gemini | Semi — RF classifies, Gemini enriches | Threshold tuning |
| Quality Narrative | Gemini Pro | Full — economic impact story | None |
| Route Comparison | Gemini Pro | Full — pros/cons summary | Read output |
| Disruption Alert | Gemini + Rule engine | Semi — rules trigger, Gemini writes alert | Approve action |
| Impact Simulation Text | Gemini Pro | Full — narrates cascade effects | None |
| UI Code Generation | Lovable (LLM) | Full — entire frontend from prompt | Iterate on prompt |

## 6.2 Gemini Prompt Templates

### Template 1: Decision Explanation

> *You are a supply chain intelligence system. Explain the following routing decision to a logistics manager in 3-4 clear sentences. Be specific about numbers.*
>
> `Route A: {route_a_name} | ETA: {route_a_eta}h | Risk: {route_a_risk}/100 | Quality at arrival: {route_a_quality}%`
>
> `Route B: {route_b_name} | ETA: {route_b_eta}h | Risk: {route_b_risk}/100 | Quality at arrival: {route_b_quality}%`
>
> `Decision: {decision} | Reason: {primary_factor}`
>
> `Cargo: {cargo_type} | Current quality: {current_quality}% | Disruption: {disruption_desc}`
>
> *Explain the decision, what was sacrificed, and what was preserved.*

### Template 2: Quality Degradation Narrative

> *You are a perishable goods expert. Translate this quality data into a business impact statement for a produce buyer.*
>
> `Cargo: {cargo_type} ({weight_kg}kg) from {origin} to {destination}`
>
> `Current quality: {quality_score}% | Status: {status}`
>
> `Delay added: {delay_hours}h | Estimated value loss: {loss_pct}%`
>
> *In 2-3 sentences: describe the quality risk, estimated market value impact, and recommended action.*

## Template 3: Mock Data Generation

> *Generate a realistic supply chain shipment scenario as valid JSON. Return ONLY JSON, no explanation.*
>
> Origin: {origin_city} | Destination: {dest_city} | Cargo: {cargo_type}
>
> JSON schema: { shipment_id, origin, destination, cargo_type, weight_kg, deadline_hours,
>
> routes: [{name, waypoints[], distance_km, base_eta_hrs, risk_factors[]}],
>
> weather_condition, congestion_level, current_temp_celsius }

## Template 4: Impact Cascade Narrative

> *Explain the supply chain cascade impact of this disruption to a logistics director. Be concise and action-oriented.*
>
> Disruption: {disruption_type} at {node_name} | Duration: {delay_hours}h
>
> Affected nodes: {affected_nodes} | Downstream shipments impacted: {shipment_count}
>
> Estimated total delay across network: {total_delay_hrs}h | Economic exposure: ${economic_value}
>
> *Provide: (1) What happened (2) What will be affected (3) Recommended immediate action*

# 7. Sample Dataset — Mock Shipment (Avocado: Kenya → India)


```
{
  "shipment_id": "SHP-2024-001",
  "cargo_type": "avocados",
  "origin": "Mombasa, Kenya",
  "destination": "Mumbai, India",
  "weight_kg": 2000,
  "deadline_hours": 72,
  "current_temp_celsius": 9,
  "routes": [
    { "name": "Route A — Direct Sea",
      "waypoints": ["Mombasa","Colombo Port","Mumbai"],
      "distance_km": 4800, "base_eta_hrs": 60,
      "risk_factors": ["monsoon_season", "port_congestion"],
      "risk_score": 65, "quality_at_arrival": 28 },
    { "name": "Route B — Via Dubai Air+Sea",
      "waypoints": ["Mombasa","Dubai","Mumbai"],
      "distance_km": 3200, "base_eta_hrs": 38,
      "risk_factors": ["airport_handling"],
      "risk_score": 28, "quality_at_arrival": 64 }
  ],
  "decision": "REROUTE_TO_B",
  "quality_status": "spoiled_if_A_acceptable_if_B"
}
```

# 8. Step-by-Step Implementation Plan (48–72 Hours)

## Day 1 (Hours 0–24): Core Infrastructure

| Time | Phase | Tasks | Deliverable |
|---|---|---|---|
| 0–3h | Setup | GCP project, Firebase, enable APIs (Maps, Gemini), repo structure | Working GCP project |
| 3–7h | Backend skeleton | FastAPI boilerplate, Pydantic schemas, Firestore connection, health check | API running locally |
| 7–12h | Risk + Quality engines | Rule-based risk scorer, quality decay formula, /predict-risk, /predict-quality endpoints | 2 working APIs |
| 12–17h | Graph + Routing | NetworkX graph, Dijkstra routing, Google Maps integration, /optimize-route | Route comparison working |
| 17–22h | Gemini integration | gemini.py client, all 4 prompt templates, /explain endpoint | Explanations working |
| 22–24h | Mock data + disruption | Mock generator, /inject-disruption endpoint, Firestore persistence | End-to-end flow works |

## Day 2 (Hours 24–48): Frontend & Integration

| Time | Phase | Tasks | Deliverable |
|---|---|---|---|
| 24–28h | Lovable UI gen | Run Lovable prompt, get base UI, connect to backend APIs | UI shell live |
| 28–33h | Map integration | Google Maps component, route polylines, node markers, real-time update | Map with routes |
| 33–38h | Quality chart | Recharts quality decay curve, time-series quality score, route comparison chart | Charts working |
| 38–43h | Decision + Gemini panel | Decision display, Gemini explanation rendering, disruption injection button | Full demo flow |
| 43–46h | Deployment | Docker build, Cloud Run deploy, Firebase Hosting deploy, domain config | Live on cloud |
| 46–48h | Demo polish | Demo script, sample data preloaded, edge cases handled, slide deck | Ready to present |

# 9. Cloud Deployment Guide (Google Only)

## 9.1 Cloud Run — Backend

- Create Dockerfile in /backend with Python 3.11-slim base
- Build: gcloud builds submit --tag gcr.io/{PROJECT_ID}/supply-chain-api
- Deploy: gcloud run deploy supply-chain-api --image gcr.io/{PROJECT_ID}/supply-chain-api --platform managed --region us-central1 --allow-unauthenticated
- Set env vars: GEMINI_API_KEY, GOOGLE_MAPS_API_KEY, FIRESTORE_PROJECT_ID

## 9.2 Firebase Hosting — Frontend

- npm run build (Vite output to /dist)
- firebase init hosting → select /dist as public directory
- firebase deploy — live in 60 seconds on firebase.app subdomain

## 9.3 Firestore — Database

- Collections: shipments, routes, risk_events, quality_logs, decisions
- Enable Firestore in Native mode in Firebase Console
- Security rules: allow read/write if authenticated (use Firebase anonymous auth for demo)

## 9.4 Required Google APIs to Enable

- Maps JavaScript API (frontend map rendering)
- Directions API (route fetching)
- Distance Matrix API (ETA computation)
- Gemini API (via Google AI Studio — get free API key)
- Cloud Run API, Cloud Build API, Artifact Registry API
- Firestore API (auto-enabled with Firebase)

# 10. Lovable Prompt — Full Demo UI

> *Copy the entire prompt below into Lovable.dev to generate the frontend. This is optimized to produce a production-quality, demo-ready UI in one generation.*

**LOVABLE PROMPT (Copy verbatim):**

```
Build a full-stack React web application called "ChainGuard AI" — an AI-
powered supply chain intelligence dashboard for perishable goods
logistics. Use Tailwind CSS for all styling. The app should feel like a
premium operations dashboard with dark-mode-friendly colors (slate-900
background, white/slate-100 text, blue-500 accents).
  == LAYOUT ==
Top navigation bar: Logo 'ChainGuard AI' on left, status pill showing
'LIVE MONITORING' in green on right, with a timestamp.
Left sidebar (240px): Shipment list showing 2 shipments — 'SHP-001:
Avocados Kenya→India' and 'SHP-002: Mangoes Peru→UK'. Each item shows
cargo type emoji, status badge (Active/At Risk), and click-to-select.
Main content area (3 panels):
  == PANEL 1: MAP VIEW (top, full width) ==
Show a simulated map area (use a styled div with a map-like gradient
background in blues and greens). Display two route lines as SVG
polylines:
- Route A (dashed red line): Mombasa → Colombo → Mumbai
- Route B (solid green line): Mombasa → Dubai → Mumbai
Each route has a labeled badge showing: Route A (60h, Risk: 65, Quality:
28%) and Route B (38h, Risk: 28, Quality: 64%). Show animated pulsing
dots at Mombasa (origin) and Mumbai (destination). Add a 'RECOMMENDED'
badge on Route B.
  == PANEL 2: METRICS ROW (3 cards side by side) ==
Card 1 — Quality Score: Large number showing '64%' in green. Below it a
mini horizontal bar chart showing quality degradation from 100% to 64%
over 38 hours. Label: 'Avocado Quality at Arrival (Route B)'. Status
badge: ACCEPTABLE.
Card 2 — Risk Score: Number '28' in green (or '65' in red for Route A).
Show a circular progress ring. Label: 'Risk Score'. Below: small list of
risk factors as colored tags (e.g. 'Airport Handling' in yellow).
Card 3 — Decision: Large text 'REROUTE TO B' in a green bordered box.
Below: 'Quality preserved: +36% vs Route A | Time saved: 22h'. A pulsing
green dot labeled 'AUTO-DECIDED'.
  == PANEL 3: BOTTOM ROW (2 sections) ==
Left section (60% width) — Quality Timeline Chart:
Use Recharts AreaChart. X-axis: Hours (0 to 72). Y-axis: Quality % (0 to
100). Show 2 area lines:
- Route A line: starts at 100, drops sharply to 28 at hour 60 (red area,
dashed)
- Route B line: starts at 100, drops gradually to 64 at hour 38 (green
area, solid)
Add horizontal dashed lines at y=75 (labeled 'Fresh') and y=40 (labeled
'Spoilage Risk') in yellow/red. Add a vertical reference line at x=38
labeled 'Route B Arrival'.
Right section (40% width) — Gemini AI Explanation Panel:
Styled like an AI chat bubble. Header: 'Gemini AI Explanation' with a
```

```
sparkle icon. Show a loading state initially (animated typing dots).
After 2 seconds, display this text in a styled card:
  "Route B via Dubai is recommended to preserve avocado quality. Route A
adds 22 additional hours and has a 65/100 risk score due to monsoon
season and port congestion at Colombo, resulting in only 28% quality at
arrival — below the spoilage threshold. Route B delivers the cargo in 38
hours with 64% quality preserved, avoiding the high-risk sea corridor.
Rerouting now saves approximately 37% of cargo value."
  Below the explanation, show 3 key factor badges: 'Time: -22h' (green),
'Risk: -37pts' (green), 'Quality: +36%' (blue).
  == INTERACTIVE CONTROLS ==
 Add a prominent red button 'Inject Disruption' in the top right of the
map panel. When clicked:
 1. Show a toast notification: 'Port delay injected at Colombo: +18
hours'
 2. Animate Route A metrics worsening: quality drops to 8%, risk
increases to 82
 3. Flash the decision card with 'AUTO-REROUTED' in pulsing red, then
transition to green
 4. Update the Gemini explanation panel to show a new explanation about
the disruption
 5. Show a new impact simulation alert: 'Delay at Colombo impacts 2
downstream warehouses'
  Also add a 'Reset Demo' button that restores all values to initial
state.
  == TECHNICAL REQUIREMENTS ==
 - Use useState and useEffect for all state management
 - Use Recharts for all chart components
 - Use Framer Motion for card entrance animations and disruption
transitions
 - Use lucide-react for icons (Truck, AlertTriangle, Zap, Shield, MapPin,
ChevronRight)
 - All mock data hardcoded in a data.js file for easy backend swapping
later
 - Add a subtle animated gradient background on the map section
 - Every metric card should have a smooth counter animation on load
 - The overall look should be dark, premium, and data-dense — like a
Bloomberg Terminal crossed with Google Maps
 - Single page app, no routing needed, responsive down to 1024px wide
```

> *After generating in Lovable: (1) Download the code, (2) Replace hardcoded data with API calls to your FastAPI backend, (3) Add real Google Maps component using @react-google-maps/api, (4) Deploy frontend to Firebase Hosting. The UI-to-backend connection should take under 2 hours.*

# 11. Alternatives Summary — Quick Reference

For every major component, here are the alternatives if primary choices fail during the hackathon:

| Feature | Primary Choice | Alt 1 (Easy swap) | Alt 2 (If both fail) |
|---|---|---|---|
| LLM Explanation | Gemini Pro | GPT-4o (OpenAI key) | Hardcoded templates |
| Maps | Google Maps API | Mapbox GL JS | Static SVG map mockup |
| Backend | FastAPI (Python) | Express.js (Node) | Serverless Functions |
| Database | Firestore | Supabase (Postgres) | localStorage (demo only) |
| Frontend Gen | Lovable.dev | v0.dev (Vercel) | Manual React coding |
| Risk Model | Random Forest (sklearn) | Logistic Regression | Pure rule-based scorer |
| Weather Data | OpenWeatherMap API | wttr.in (free, no key) | Hardcoded mock values |
| Graph Routing | NetworkX + Dijkstra | Custom Dijkstra in JS | Hardcoded 2-route mock |
| Hosting | Firebase + Cloud Run | Vercel + Railway | ngrok for local demo |

> *Hackathon survival rule: Always have your mock data version working first. Real APIs are extras. The Gemini explanation + quality chart + disruption simulation are the three 'wow' moments — protect those at all costs.*

**ChainGuard AI — Built for Speed, Designed for Impact**

*Track → Analyze → Predict → Simulate → Decide → Explain*